설비 고장을 예측하는 AI 데이터 분석

설비 상태 변화 패턴 분석

다중센서 패턴 분석

posco

CONTENTS

목차

01 시간 인덱싱과 리샘플링

02 다중센서 비교

03 상관관계와 종합 분석

posco

posco

01

시간 인덱싱과 리샘플링

timestamp 데이터를 인덱스로 다루고 시간 단위로 묶어 요약하기

posco

cycle과 timestamp의 차이

시간을 표현하는 두 방식 — 횟수(cycle)와 날짜·시각(timestamp)

01 · 두 가지 방식

시계열의 시간은 횟수 또는 날짜·시각

cycle은 작동 횟수, timestamp는 실제 날짜·시각으로 표현

02 · 표현의 차이

cycle은 순서만, timestamp는 언제인가까지

1·2·3 순서만 vs 2024-01-01 09:00 같은 실제 시각

03 · 도구의 차이

리샘플링·시간 집계는 timestamp에서만 작동

cycle엔 하루라는 개념이 없어 시간 단위 묶기가 불가능

cycle과 timestamp 비교

두 방식은 우열이 아니라 쓸 수 있는 도구가 다를 뿐

Q1. cycle 기반은 무엇인가

작동 횟수만 세는 방식

1, 2, 3 순서만 의미
이동평균·상관 등 **순서 기반 도구**에 사용 가능

Q2. timestamp 기반은 무엇인가

실제 날짜·시각으로 표현

오전 9시·어제처럼 시간 질문 가능
리샘플링·시간 집계에 사용

사이클에 시간 간격을 정해주면 timestamp로 변환 가능 — 이번 시간에서 바로 활용

posco

날짜형 변환 샘플이 필요한 이유

C-MAPSS는 cycle 기반이라 시간 도구 연습을 위해 가상의 시각을 입혀 변환 샘플 제작

리샘플링·시간 인덱싱은 **timestamp에서만 작동**
실무 데이터 대비 꼭 배워야 함

그런데 C-MAPSS는 **cycle 기반**이라 바로 연습 불가
주력 데이터에 날짜·시각이 없는 상황

해법은 사이클에 **가상의 날짜·시각**을 입혀 변환 샘플 만들기
학습 편의용 변환, 실무에선 원래 시각을 그대로 사용

posco

사이클에 가상 시각 입히기

1 사이클을 1시간으로 약속하고 사이클마다 1시간씩 부여 — 3단계 변환

STEP 1

1 사이클 = 1시간

학습 편의를 위한 약속. 사이클 간격을 시간 단위로 정의

STEP 2

2024-01-01부터 1시간씩

시작 시각을 기준으로 사이클마다 한 시간씩 더해 시각 부여

STEP 3

192사이클 = 192시간

1번 엔진 기준 8일치 데이터 완성. 일평균·6시간 평균 연습 가능

이 변환 샘플은 학습용. 실무에선 데이터에 원래 있는 진짜 시각을 그대로 사용

시간을 인덱스로 만들기

시간 도구를 쓰려면 형식 변환과 인덱스 지정 두 가지 준비가 필요

STEP 1

글자 → 시간 형식

`to_datetime`으로 텍스트 날짜를 진짜 시간 형식으로 변환

object/str → datetime64

STEP 2

set_index로 시간 지정

시간 열을 인덱스로. 날짜로 자르기·시간으로 묶기가 쉬워짐

DatetimeIndex

STEP 3

자료형 확인 습관

변환 후 datetime으로 바뀌었는지 출력해 확인

dtype 점검

posco

변환 전후 비교

인덱스 자료형이 바뀌면 시간 단위 조작이 가능 — 번호에서 시각으로

관점	변환 전	변환 후
인덱스 값	0, 1, 2 같은 번호	2024-01-01 00:00 같은 실제 시각
시간 계산	불가	날짜·시간 기준 조작 가능
시간 열의 위치	일반 열에 위치	인덱스 자리로 이동 — 사라진 게 아님

set_index 후 시간 열이 안 보여 당황하지 않기 — 인덱스 자리로 이동했을 뿐, 더 중요한 자리

시간 구간 잘라보기

DatetimeIndex가 되면 날짜만 적어 그 구간을 통째로 추출

시간이 인덱스가 되면 **날짜만 적어** 그 구간을 잘라낼 수 있음

DatetimeIndex의 가장 큰 장점

하루는 `.loc`에 날짜 하나, 기간은 시작일과 끝일을 **콜론으로 연결**

.loc["2024-01-01"] · .loc["2024-01-02":"2024-01-03"]

번호로 몇 번째 행인지 셀 필요 없이 **사람이 생각하는 방식** 그대로

cycle 기반과의 결정적 차이 — 1월 2일부터 3일까지처럼 자름

구간 잘라보기의 활용

넓게 훑어 의심 구간을 찾고 좁혀서 자세히 보는 흐름이 기본

01 · 하루·기간

날짜 하나 또는 시작·끝일로 추출

하루치는 24행, 이틀치는 48행처럼 시간 단위로 정확히 잘림

02 · 집중 분석

고장 의심 기간만 떼어 자세히

지난주만, 고장 난 날 전후만 — 특정 기간에 집중하는 실무 패턴

03 · 잘린 구간

잘라낸 구간도 시계열

첫 시간의 이동평균·변화량을 다시 적용 가능

posco

resample 시간 단위 집계

timestamp 분석의 핵심 — 잘게 측정된 데이터를 큰 시간 단위로 묶어 요약

STEP 1

묶을 시간 단위 지정

D = 하루, h = 시간, 6h = 6시간

```
resample("D")
```

STEP 2

집계 방법 지정

.mean 평균 또는 .max 최댓값

```
.mean() / .max()
```

STEP 3

개수가 확 줄어듦

8일치를 D로 묶으면 8개. rolling은 겹쳐 계산해 개수 그대로

192 → 8

설비에선 평균보다 최댓값이 중요할 때가 많음 — 평균과 최댓값을 함께 보기

리샘플링의 효과와 주의

묶는 단위는 목적에 맞게 — 첫 시간의 창 크기 고민이 여기서도 반복

192시간 데이터를 하루로 묶으면 **8개** — 흔들림에 묻힌 흐름이 선명

촘촘할 때 안 보이던 642 → 650 상승이 또렷이 드러남

리샘플링 결과에 **이동평균을 다시 적용** 가능 — 도구를 단계적으로 쌓음

요약본의 추세를 한 번 더 부드럽게 보기

단위가 너무 크면 변화가 **뭉개지고** 너무 작으면 요약 효과 적음

목적에 맞게 — 한 시간 사이 급변이 일평균에 녹아 사라질 수 있음

실습 1. 시간 인덱싱과 슬라이싱

글자 날짜를 시간 형식으로 바꿔 인덱스로 올리고 자르기

목표

글자 날짜를 시간 형식으로 바꿔 인덱스로 올리고 날짜로 구간 자르기

단계

· 시각 열의 자료형을 확인하고 시간 형식으로 변환

· 시간 열을 인덱스로 올리기

· 하루·기간을 날짜로 잘라 행 수 확인

예상 결과

자료형이 시간으로 바뀌고, 하루 24행·사흘 72행

posco

실습 1. 시간 인덱싱과 슬라이싱 정답

글자 날짜를 시간 형식으로 바꿔 인덱스로 올리고 자르기

CODE · 실습 1

```
import pandas as pd
ts = pd.read_csv('cmapss_unit1_timestamp.csv')
ts['timestamp'] = pd.to_datetime(ts['timestamp'])
ts = ts.set_index('timestamp')
print(len(ts.loc['2024-01-02']))                # 24
print(len(ts.loc['2024-01-03':'2024-01-05']))   # 72
```

실습 2. 리샘플링 집계와 시간 이동통계

시간 단위로 대표값을 구하고 일별 이동평균으로 추세 보기

목표

시간 단위로 리샘플링해 대표값을 구하고 일별 이동평균으로 추세를 보기

단계

· 하루 단위 평균과 6시간 단위 평균을 각각 구하기

· 각 결과의 개수로 단위 차이를 확인

· 일별 평균에 이동평균을 걸어 추세를 매끄럽게

예상 결과

하루 평균 8개·6시간 평균 32개, 일별 값이 상승 추세

posco

실습 2. 리샘플링 집계와 시간 이동통계 정답

시간 단위로 대표값을 구하고 일별 이동평균으로 추세 보기

CODE · 실습 2

```
daily = ts['sensor_3'].resample('D').mean()
print(len(daily), daily.round(1).tolist())
# 8  [1592.1, 1590.9, 1593.3, 1597.8, 1603.7, 1610.6, 1614.1, 1628.5]
print('6시간 개수:', len(ts['sensor_3'].resample('6h').mean()))   # 32
daily_ma = daily.rolling(3).mean()
```

02

다중센서 비교

정규화로 눈금을 맞추고 여러 센서를 한 화면에서 함께 읽기

posco

여러 센서를 함께 볼 때의 문제

데이터가 아니라 그리는 방식의 문제 — 스케일 차이가 만든 시각적 왜곡

설비 진단은 **여러 센서를 함께 봐야** 전체 상태가 보임

한 부품 이상이 여러 센서에 영향 — 종합 관점이 필수

640짜리와 8짜리 센서를 같이 그리면 큰 값이 **화면을 독차지**

640짜리가 위쪽을 가득 채워 다른 센서를 가림

8짜리 센서는 바닥에 납작하게 깔려 변화가 안 보임 — **스케일 문제**

8 근처에서 출렁여도 640까지 펼친 축에선 직선처럼 보임

눈금 맞추기가 필요한 이유

센서가 많아질수록 정규화는 선택이 아닌 필수

문제 01

큰 값이 축 독차지하면
작은 센서 변화 사라짐

키 180과 몸무게 70을 같은 막대로 그리는 셈

문제 02

따로 그리면
같은 시점 비교 어려움

번갈아 보며 시점 맞추기는 번거롭고 정확도 떨어짐

해법

같은 잣대로 변환하는
눈금 맞추기 — 정규화

각 센서의 값 범위를 동일한 기준으로 변환

posco

min-max 정규화

스케일 문제를 푸는 가장 쉬운 방법 — 각 센서를 0~1 사이로 펴기

각 센서의 값을 **0과 1 사이**로 변환
최솟값 0, 최댓값 1

각 값에서 **최솟값을 빼고(최댓값-최솟값)으로 나누기**

```
norm = (x - x.min()) / (x.max() - x.min())
```

640~660 센서라면 640은 **0**, 660은 **1**, 650은 **0.5**
범위 중 어디쯤인지 0~1 비율로 표현

posco

정규화의 특징

크기를 없애고 변화의 모양만 남김 — 추세 방향은 보존, 단위는 비율로

01 · 모양 남기기

값의 크기 없애고 변화의 모양만

640짜리든 8짜리든 정규화 후 모두 0~1 사이 — 공정한 비교 가능

02 · 추세 보존

정규화해도 추세 방향은 그대로

올라가던 센서는 정규화 후에도 올라감 — 모양 보존

03 · 사용 한계

0~1 비율로 바뀌므로 비교 목적에만

실제 값이 궁금할 땐 원본 사용 — 이상치 있으면 정규화 전 점검

이상치가 하나 섞이면 그게 최댓값이 돼 나머지가 눌림 — 우리 데이터는 깨끗

다중센서 한 화면 비교

정규화 먼저, 이동평균 나중 — 역할 분담이 핵심

STEP 1

비교 센서 골라 정규화

비교할 센서들을 골라 모두 0~1로 변환. 크기 맞추기

STEP 2

이동평균으로 다듬기

각 센서에 이동평균을 씌워 흔들림 다듬기. 선 엉킴 방지

STEP 3

한 그래프에 겹쳐 그리기

다듬은 센서들을 한 화면에 겹쳐 그리고 범례로 구분

너무 많이 올리면 엉키니 3~4개로 추리기

posco

비교 그림 읽는 법

함께 vs 반대를 눈으로 보는 것이 다중센서 분석의 첫걸음

여러 정규화 이동평균선에서 **함께 가는 무리**와 반대로 가는 센서 찾기
한 화면에 여러 센서가 그려질 때의 기본 시선

함께 올라가는 센서들은 **같은 현상**을, 반대로 가는 센서는 **다른 측면**을 반영
한 부품이 나빠지면 연결된 여러 센서가 함께 움직임

여러 센서가 같은 시점에 함께 변하면 **진짜 변화일 가능성** 상승
상관관계의 눈으로 보는 버전 — 다음 파트에서 숫자로 측정

흔한 실수 — 정규화 빼먹고 바로 그리기. 직선처럼 보이면 정규화 여부 확인

실습 3. 다중센서 정규화와 겹쳐 비교

여러 센서를 0~1로 맞춰 같은 눈금에서 함께 비교

목표

여러 센서를 0~1로 정규화해 같은 눈금에서 함께 비교

단계

· 세 센서를 최소·최대 기준으로 0~1로 정규화

· 정규화 후 최솟값·최댓값이 0과 1인지 확인

· 이동평균을 걸어 겹쳐 그릴 준비

예상 결과

각 센서의 최솟값 0·최댓값 1로 눈금이 맞춰짐

posco

실습 3. 다중센서 정규화와 겹쳐 비교 정답

여러 센서를 0~1로 맞춰 같은 눈금에서 함께 비교

CODE · 실습 3

```
df = pd.read_csv('cmapss_fd001_sample.csv')
u2 = df[df['unit_number'] == 2].copy().reset_index(drop=True)
cols = ['sensor_2', 'sensor_7', 'sensor_11']
norm = (u2[cols] - u2[cols].min()) / (u2[cols].max() - u2[cols].min())
print(norm.min().tolist(), norm.max().tolist())   # [0,0,0] [1,1,1]
norm_ma = norm.rolling(window=10).mean()
```

실습 4. 다중센서 변화 폭 비교

센서별 초반·후반을 비교해 변화 폭 큰 감시센서 찾기

목표

정규화한 여러 센서의 초반·후반 값을 비교해 변화 폭이 큰 센서를 찾기

단계

· 여섯 센서를 정규화하고 이동평균을 걸기

· 각 센서의 초반과 후반 평균을 구하기

· 초반·후반 차이로 변화 폭이 큰 감시 센서 선별

예상 결과

sensor_2는 크게 상승, sensor_7은 하락 — 변화 폭 큰 센서 부각

실습 4. 다중센서 변화 폭 비교 정답

센서별 초반·후반을 비교해 변화 폭 큰 감시센서 찾기

CODE · 실습 4

```
sensors = ['sensor_2', 'sensor_3', 'sensor_4', 'sensor_7', 'sensor_11', 'sensor_15']
n6 = (u2[sensors] - u2[sensors].min()) / (u2[sensors].max() - u2[sensors].min())
n6ma = n6.rolling(window=10).mean()
for c in ['sensor_2', 'sensor_7', 'sensor_11']:
    e = n6ma[c].iloc[10:40].mean(); l = n6ma[c].iloc[150:].mean()
    print(c, round(e, 2), round(l, 2), round(l - e, 2))
# sensor_2 0.07 0.53 +0.46,  sensor_7 0.9 0.49 -0.41,  sensor_11 0.13 0.39 +0.26
```

03

상관관계와 종합 분석

숫자로 재는 관계, 신호 종합으로 짚는 이상 구간, 정비 리포트로 번역

센서 간 상관관계

어림짐작을 객관적 숫자로 — 상관계수

상관관계는 두 센서가 **함께 변하는 정도**를 하나의 숫자로 표현

함께 가는 것 같아요 → 상관계수 0.85로 강하게 함께 갑니다

눈대중이 아니라 객관적 숫자 — **상관계수**

주관적 인상을 정량적 진술로 바꿈

판다스의 `.corr()`로 모든 센서 쌍의 상관을 표(**상관행렬**)로 제작

한 번의 호출로 N×N 표 완성

posco

상관관계의 쓰임

같은 현상 파악, 센서 줄이기, 관계 변화 감지까지 — 세 가지 용도

01 · 같은 현상

강하게 상관된 무리
= 같은 현상 반영

같은 부품 이상이 여러 센서에 영향 → 강한 상관 무리 식별

02 · 센서 줄이기

거의 같이 움직이면
하나만 봐도 됨

0.99처럼 거의 같은 정보 → 대표만 골라 분석 간결화

03 · 관계 변화

평소 관계가 깨지면
그 자체가 이상 신호

함께 가던 둘이 갑자기 따로 놀 때 — 이상 감지로 확장

상관계수 읽는 법

상관계수는 -1과 1 사이 — 부호는 방향, 절댓값은 강도

01 · 부호

양수면 같은 방향
음수면 반대 방향

양수: 함께 오르고 함께 내림
음수: 한쪽 오르면 다른 쪽 내림 — 온도↑·압력↓

02 · 절댓값

1에 가까울수록 강한 관계
0에 가까울수록 약한 관계

부호를 떼고 크기만 보면 강도
보통 **0.7 이상**이면 강한 상관

posco

상관계수 해석 예시

상관행렬의 대각선은 항상 1 — 절반만 봐도 충분

0.94는 강하게 반대로(절댓값 0.94 강함, 부호 음), +0.85는 강하게 함께
온도↑ 시 압력이 거의 그대로 내려가는 강한 반대 관계

상관행렬의 **대각선은 항상 1** — 자기 자신과의 상관
무시하고 대각선 바깥만 분석

대각선 기준 위아래가 같아 **절반만** 봐도 됨
대칭 행렬 — A↔B 상관과 B↔A 상관은 동일

단, 상관계수 0 = 관계 없음이 아님 — 곡선 관계는 못 잡음. 그래프로도 확인 필요

posco

상관행렬을 heatmap으로

숫자 표를 색으로 변환 — 6×6의 36칸도 한눈에

STEP 1

상관행렬 구하기

비교할 센서들의 상관행렬을 .corr()로 구함

STEP 2

heatmap에 넣어 색칠

양의 상관은 빨강, 음의 상관은 파랑

STEP 3

옵션으로 다듬기

숫자 표시·색 설정·중심값 옵션

진한 빨강 = 강하게 함께, 진한 파랑 = 강하게 반대 — 핵심 관계만 즉시 파악

posco

heatmap 읽는 법

복잡한 관계를 색 패턴으로 압축하는 heatmap의 힘

진한 빨강 칸은 강하게 함께 가는 센서 쌍, 진한 파랑 칸은 강하게 반대
색만 보면 됨 — 단순한 읽기 규칙

흰색에 가까우면 관계가 약함 — **대각선은 항상 진한 색(1)**
대각선은 자기 자신과의 상관이라 무시

센서가 많아질수록 진한 색 **덩어리**로 함께 묶이는 무리가 한눈에 드러남
사람 눈은 숫자보다 색에 빠르게 반응 — 패턴 인식

posco

상관은 인과가 아니다

함께 변한다 ≠ 원인이다 — 가장 흔한 해석 실수

01 · 흔한 실수

함께 변해도 한쪽이 다른 쪽의 원인은 아님

"함께 변한다"를 "하나가 원인이다"로 착각 금지

02 · 실제 원인

제3의 공통 원인이 둘 다에

부품 열화가 온도와 압력 둘 다에 영향. 아이스크림과 익사 = 여름

03 · 올바른 자세

상관은 관계 발견의 출발점이지 결론 아님

"왜 그럴까" 질문의 시작 — 해석은 설비 도메인의 몫

실습 5. 다중센서 상관 heatmap

여러 센서의 상관행렬을 색으로 그려 관계 파악

목표

여러 센서의 상관행렬을 구해 heatmap으로 함께 움직이는 센서를 파악

단계

· 여섯 센서의 상관행렬을 계산

· 히트맵으로 색과 숫자를 함께 표시

· 진한 칸으로 강한 양·음 관계를 읽기

예상 결과

sensor_2와 sensor_7이 강한 음(-0.96), sensor_2와 sensor_11이 양(0.84)

posco

실습 5. 다중센서 상관 heatmap 정답

여러 센서의 상관행렬을 색으로 그려 관계 파악

CODE · 실습 5

```
import seaborn as sns
import matplotlib.pyplot as plt
corr = u2[sensors].corr()
print('s2-s7:', round(corr.loc['sensor_2', 'sensor_7'], 2),
      's2-s11:', round(corr.loc['sensor_2', 'sensor_11'], 2))   # -0.96 / 0.84
sns.heatmap(corr, annot=True, cmap='coolwarm', center=0, fmt='.2f')
plt.tight_layout(); plt.show()
```

실습 6. 상관 높은 센서쌍 추출

대각선을 빼고 가장 강한 음·양의 센서쌍 추출

목표

상관행렬에서 대각선을 빼고 가장 강한 음·양의 센서쌍을 추출

단계

· 상관행렬에서 자기 자신 대각선을 제외

· 센서쌍을 상관값 순으로 정렬

· 가장 강한 음의 쌍과 양의 쌍을 확인

예상 결과

음의 최강쌍 sensor_2·sensor_7(-0.955), 양의 최강쌍 sensor_2·sensor_11(0.839)

posco

실습 6. 상관 높은 센서쌍 추출 정답

대각선을 빼고 가장 강한 음·양의 센서쌍 추출

CODE · 실습 6

```
import numpy as np
pairs = corr.where(~np.eye(len(corr), dtype=bool)).unstack().dropna()
pairs = pairs[pairs.index.map(lambda x: x[0] < x[1])].sort_values()
print('음:', pairs.index[0], round(pairs.iloc[0], 3))    # ('sensor_2','sensor_7') -0.955
print('양:', pairs.index[-1], round(pairs.iloc[-1], 3))   # ('sensor_11','sensor_2') 0.839
```

시계열 분해 개념

시계열을 세 조각으로 쪼개 보기 — 합치면 원본, 나누면 따로

01 · 추세

한 방향으로
길게 흘러가는 흐름

첫 시간에서 본 그 추세 — 큰 흐름

02 · 주기

일정 간격으로
반복되는 패턴

하루·일주일 주기 — 낮 높고 밤 낮음

03 · 잔차

추세·주기로 설명 안 되는
불규칙한 나머지

셋을 합치면 원본, 나누면 각각 따로

시계열 분해가 유효한 경우

주기성이 뚜렷한 데이터에서 진가 — C-MAPSS는 선택 주제로

주기성이 **뚜렷한** 데이터(시간대별 전력·요일별 매출)에서 진가 발휘
하루나 일주일처럼 명확한 반복이 있어야 의미

C-MAPSS는 한 방향 열화만 있고 **주기가 약해** 주기 요소는 큰 의미 없음
그래서 본 강의에서는 선택 주제로 다룸

잔차가 유난히 커지는 구간은 평소 패턴으로 설명 안 되는 **이상 신호**
이상 탐지로 활용 가능

posco

seasonal_decompose 시연

도구는 나눠주지만 무엇으로 나눌지는 사람이 정함

STEP 1

불러오기

statsmodels의 seasonal_decompose 불러오기

STEP 2

시계열·주기 지정

한 시간 간격·하루 주기면 24 — 반복 간격

STEP 3

자동 분해·그리기

추세·주기·잔차로 자동 분해해 그리기

주기는 사람이 지정해야 함 — 어떤 반복이 있을지는 설비를 아는 사람이 결정

분해 결과 읽는 법

가장 흥미로운 칸은 잔차 — 예상 가능한 부분을 걷어낸 이상 신호

결과는 보통 **4칸** — 위에서부터 원본, 추세, 주기, 잔차
시각적으로 분리된 네 단계의 그림

네 칸을 더하면 다시 원본 — **위→아래**로 전체에서 나머지로 읽기
분해의 합 = 원본의 보존 법칙

가장 흥미로운 칸은 **잔차** — 예상 가능한 부분을 걷어낸 이상 신호
첫 시간의 잔차 기반 변화점과 발상이 통함

이상 의심 구간 판단 기준

여러 신호가 한 구간에 동시에 나타나면 강하게 의심

01 · 추세 꺾임

평평하던 이동평균선이 기울기 시작

한 방향으로 기울기 시작하는 구간 — 상태 변화의 시작 신호

02 · 변동성 증가

이동표준편차가 갑자기 커짐

불안정해지는 구간 — 단순 노이즈가 아닌 패턴 변화

03 · 변화점 집중

변화점이 한 구간에 여럿 몰림

같은 시점에 여러 급변이 발생 — 우연이 아닐 가능성

04 · 관계 흐트러짐

함께 가던 센서들의 관계가 깨짐

상관관계가 흔들리는 구간 — 다중 신호의 마지막 단서

posco

신호가 겹칠수록 의심이 강해진다

여러 증인의 증언이 한 시간대를 가리키면 우연일 확률 급감

한 신호만으로는 노이즈일 수 있지만 여러 신호가 겹치면 **진짜 변화일 가능성**
독립적인 신호의 일치 = 우연이 아닐 통계적 근거

각 신호가 가리키는 구간을 한 시간축에 표시하면 **겹치는 구간**이 드러남
추세 꺾임·변동성·변화점 몰림을 같은 축에 겹쳐 보기

판단에 정답표는 없음 — 후보를 좁혀 **사람이 최종 해석**
데이터는 후보를 좁히고, 결정은 도메인 전문가가

분석 결과를 정비 언어로

같은 사실인데 **행동하기 쉬운** 표현으로 바꾸는 것이 분석의 가치

❌ 분석가 표현(통계 언어)	✅ 정비 담당자 표현(설비 언어)
이동평균 상승	온도가 점진적으로 상승하고 있음
변동성 2배 증가	불안정해져 점검이 필요함
잔차 큰 변화점 발생	평소와 다른 변화가 oo 사이클부터

통계 언어를 무엇·언제·행동이 담긴 문장으로 번역해야 가치 발생

posco

정비 언어 번역의 핵심

무엇·언제·행동 세 요소로 분해 — 단정 대신 행동 유도

01 · 무엇

무엇이 관찰됐나

설비 언어로 표현

예: 온도가 점진적으로 상승하고 있음

02 · 언제

언제부터인가

구체적으로 짚기

예: 약 OO 사이클부터 (수명 약 70% 지점)

03 · 행동

무엇을 할까

행동으로 제안

예: 점검 권장, 모니터링 강화

의심이지 확정 아님 — "고장입니다" 대신 "열화가 의심되니 점검을 권장합니다"

posco

정비 판단 리포트 프레임

막막하게 쓰지 않고 정해진 틀에 채우기

STEP 1 · 관찰

데이터에서 무엇을 봤나

어떤 센서가 어떻게 변했는지 설비 언어로

STEP 2 · 구간

변화가 언제 일어났나

구체적 시점 또는 사이클로

STEP 3 · 해석

무슨 의미인가

추정 상태를 적기. 관찰은 사실, 해석은 추정

여기에 네 번째 권장(무엇을 할까)이 더해짐 — 관찰과 해석은 섞지 말 것

posco

리포트의 네 항목

반드시 행동으로 마무리 — 권장 없는 리포트는 "그래서 어쩌라는 거지"

01 · 관찰

온도 점진 상승, 흔들림 약 2배 증가

데이터 사실 — 막연한 느낌 대신 숫자

02 · 구간

후반부 약 150 사이클 이후 뚜렷

언제부터 — 구체적으로

03 · 해석

온도 상승과 불안정 동반 — 부품 열화 추정

추정 — 단정 아님

04 · 권장

정밀 점검과 모니터링 강화

행동 제안 — 반드시 포함

posco

실습 7. 종합 통합 분석

이동통계·변화점을 구하고 구간별 이상 의심 구간 도출

목표

이동통계·변화점을 계산하고 구간별로 나눠 이상 의심 구간을 도출

단계

· 이동평균·이동표준편차·변화점을 한 번에 계산

· 초반·후반 변동성과 변화점 개수를 요약

· 회차 구간별 변동성·변화점으로 이상 의심 구간 찾기

예상 결과

후반 변동성이 커지고 변화점 19개, 216~288 구간에 집중

posco

실습 7. 종합 통합 분석 정답

이동통계·변화점을 구하고 구간별 이상 의심 구간 도출

CODE · 실습 7

```
s = u2['sensor_2']
u2['ma'] = s.rolling(10).mean(); u2['std'] = s.rolling(10).std()
resid = s - u2['ma']; u2['chg'] = resid.abs() > 2 * resid.std()
print('변동성 초반/후반:', round(u2['std'].iloc[10:50].mean(), 3), round(u2['std'].iloc[150:].mean(), 3))  # 0.319 / 0.516
print('변화점:', int(u2['chg'].sum()))            # 19
for lo, hi in ((0, 72), (72, 144), (144, 216), (216, 288)):
    seg = u2[(u2.time_cycles > lo) & (u2.time_cycles <= hi)]
    print(f'  {lo}~{hi}: 변동성 {seg["std"].mean():.3f}, 변화점 {int(seg["chg"].sum())}개')
# 216~288 변동성 0.723, 변화점 18개 (후반 집중)
```

분석의 전체 여정

데이터에서 시작해 현장 행동 제안으로 끝나는 전체 여정

STEP 1 · 한 센서

추세·변동성
변화량·변화점

첫 시간 — 추세·변동성으로 상태 추적, 변화량·변화점으로 급변 포착

STEP 2 · 다중센서

시간 인덱싱·리샘플링
정규화·상관

둘째 시간 — 시간 분석, 여러 센서 비교, 센서 관계 파악

STEP 3 · 종합·번역

신호 종합·이상 구간
정비 리포트로 번역

모든 신호를 종합해 이상 의심 구간 도출, 정비 판단 리포트로 번역

도메인이 자산, AI는 도구 — 현장 전문성과 결합하면 강력한 분석가